Programação Avançada

Unidade 1: POO (Programação Orientada a Objetos)

Aula 03: POO – Namespaces e Modularização

Prof. Eduardo Corrêa Gonçalves

16/03/2026

Sumário

Introdução

Modularização

Namespaces

O que são namespaces?

A função dir()

Removendo um nome

Módulos

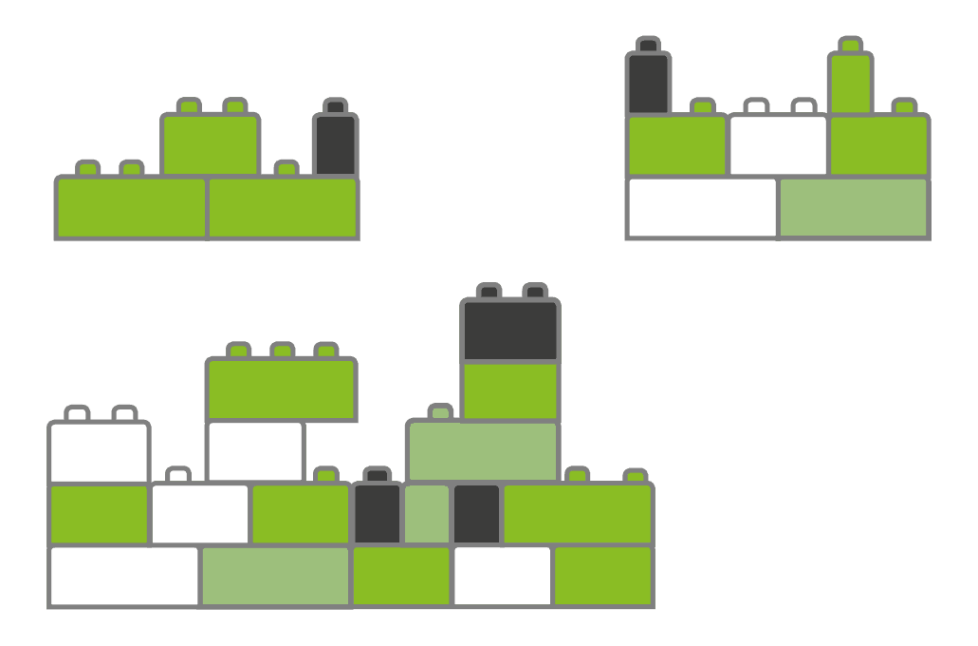
Criando Módulos

Importando Módulos

A variável de sistema __name__

Introdução (1/2)

- **Modularização**
  - Em um projeto modularizado, um sistema é quebrado em partes menores (**módulos**) que podem ser criados e testados de forma independente.
  - Trata-se de uma técnica indispensável para o desenvolvimento de quase todos os tipos de produto...
    - ... E não é diferente com os softwares!



Introdução (2/2)

- **Modularização**
  - Uma das grandes vantagens da POO é que ela naturalmente facilita a modularização do sistema.
  - Em geral, em um sistema orientado a objetos, definimos cada classe em um arquivo separado.
    - Por exemplo a classe “Musica” em “musica.py”, a classe “PlayList” em “playlist.py”, etc.
    - Ou seja, criamos um **módulo** para cada classe.
    - Se uma classe precisa ser utilizada por outra classe ou outro programa, basta importa-la com o uso do comando **import**.

Namespaces (1/8)

- **O que são namespaces?**
  - Nesta aula, apresentaremos uma breve introdução à modularização no Python.
  - Mas antes de tudo, é importante apresentar o conceito de **namespace**.
  - Trata-se de um mecanismo utilizado por diversas linguagens de programação para **evitar conflitos de nomes**.
    - **Ex.**: classes, funções ou variáveis com nomes iguais em módulos diferentes.

Namespaces (2/8)

- **O que são namespaces?**
  - Como sabemos, em um computador podemos ter arquivos com o mesmo nome, armazenados em pastas diferentes.
  - **Exemplo**:
    - "lista.xls" na pasta "c:\documentos\compras" (lista de compras)
    - "lista.xls" na pasta "c:\ence\calculo" (lista de alunos da turma de cálculo)
  - Da mesma forma, os namespaces são utilizados para permitir que o mesmo nome - por exemplo "x", "y", "inserir()", "classificar()"- possa ser utilizado em diferentes módulos.

Namespaces (3/8)

- **dir()**
  - A função **dir()** retorna uma lista dos nomes definidos em um namespace.
  - Se você abrir o interpretador Python e logo digitar **dir()**, serão listados os nomes que estão automaticamente definidos quando o interpretador inicia.

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__']
```

  - Todos os nomes acima estão disponíveis em qualquer programa Python. Conforme novos nomes forem definidos, eles passarão a aparecer nas novas chamadas da função **dir()**:

```
>>> lst = ["A","B","C"]
>>> x = 100
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'lst', 'x']
```

Namespaces (4/8)

- **Módulo __builtins__**
  - Com **dir(__builtins__)**, você terá a lista de todos os nomes do **módulo builtins**.
    - Esse é um **módulo especial**, carregado automaticamente pelo Python e contém todas as funções e objetos pré-definidos (built-in).
    - Ou seja, você não precisa importar esse módulo para usar qualquer função dele. Alguns exemplos: len(), print(), max(), ...
    - Você pode utilizar qualquer nome do builtins diretamente em seu código, sem precisar prefixar com o nome do módulo.
      - Por exemplo: pode usar len(), em vez do nome completo __builtins__.len() .

Namespaces (5/8)

- **Módulo __builtins__**

```
>>> dir(__builtins__)
```

['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '_', '__build_class__', '__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'all', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']

Namespaces (6/8)

- **Namespaces e import**
  - A função **dir()** é especialmente útil para que possamos identificar que nomes foram adicionados ao namespace após um comando **import**:

```
>>> import math
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'math']
>>>
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
>>>
```

Namespaces (7/8)

- **Removendo um nome**
  - É possível remover um nome da sua sessão utilizando o comando **del**.
    - Na verdade, esse comando serve para remover qualquer objeto da memória: variável, lista, dicionário, função... Qualquer coisa!
    - Uma vez que o objeto é removido, seu nome também é removido do namespace.
  - O código abaixo remove o nome "x"

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'lst', 'sys', 'x']
>>>
>>> del x
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'lst', 'sys']
```

Namespaces (8/8)

- **Removendo um nome**
  - Para um exemplo mais sofisticado sobre a remoção de nomes, baseado no uso do módulo ‘sys’, consulte:
    - https://stackoverflow.com/questions/3543833/how-do-i-clear-all-variables-in-the-middle-of-a-python-script

Modularização no Python (1/15)

- **import**
  - Já que falamos do **import**, esse é o principal comando para a modularização no Python.
  - Como o nome indica, é um comando que permite que um **módulo** seja importado em um programa.
  - Mas **o que é um módulo** Python?
    - Basicamente, qualquer arquivo que possua uma extensão ".py" e contenha código Python* pode ser considerado um módulo.
    - Não há nenhuma sintaxe especial que seja exigida para que você escreva um módulo.
    - Um módulo pode possuir funções, EDs (listas, dicionários, etc.), classes, enfim, qualquer objeto Python!

* Também é possível importar módulos escritos em C

Modularização no Python (2/15)

- **import**
  - Então vamos criar um módulo!
  - Digite o programa abaixo e salve com o nome “mod1.py”:

```
s = "Total eclipse of the heart"
t = (5, 10, 15, 20)

def f_teste(x):
    print('x = ', x)

class MinhaClasse:
    def __init__(self):
        print("objeto instanciado!")
```

  - Veja que o programa define **4 objetos**:
    - s (uma string), t (uma tupla), f_teste (uma função) e MinhaClasse (uma classe).

Modularização no Python (3/15)

- **import**
  - Na mesma pasta, crie e execute o programa “testa_mod1.py”
  - Esse programa acessa os objetos de “mod1.py” através do import.
    - **OBS**: o import diferencia letras maiúsculas e minúsculas (*se o nome foi salvo em minúsculo, deverá ser especificado assim*)

```
import mod1
print(mod1.s)
print(mod1.t)
mod1.f_teste(1000)
a = mod1.MinhaClasse()
```

```
>>> %Run testa_mod1.py
Total eclipse of the heart
(5, 10, 15, 20)
x =  1000
objeto instanciado!
```

Modularização no Python (4/15)

- **import - pastas**
  - Quando o interpretador executa o import do programa abaixo, ele procura por “mod1.py” em uma lista de diretórios (pastas), de acordo com a seguinte ordem:
    - Pasta corrente (a mesma onde está “testa_mod1.py”)
    - Lista de diretórios contida na variável de ambiente PYTHONPATH (varia de acordo com o Sistema Operacional).

```
import mod1
print(mod1.s)
print(mod1.t)
mod1.f_teste(1000)
a = mod1.MinhaClasse()
```

Modularização no Python (5/15)

- **import - pastas**
  - Para saber quais são essas pastas, você pode consultar a variável **sys.path**, disponível no módulo **sys**:

```
>>> import sys
>>> sys.path
['D:\\ENCE\\ProgAvancada\\aula11', 'D:\\Thonny\\python37.zip',
'D:\\Thonny\\DLLs', 'D:\\Thonny\\lib', 'D:\\Thonny',
'C:\\Users\\eduardo\\AppData\\Roaming\\Python\\Python37\\site-packages',
'D:\\Thonny\\lib\\site-packages’]
```

  - **Obs1**: conteúdo exato mostrado por sys.path depende do ambiente Python que você estiver usando, da instalação, etc.
  - **Obs2**: para simplificar, nos exemplos dessa aula colocaremos sempre os módulos e os programas que os importam dentro de uma mesma pasta.

Modularização no Python (6/15)

- **import – formas de utilização**
  - Veja que para acessar os objetos, precisamos prefixar com o nome do módulo e um ponto: *mod1.nome_do_objeto*.
    - Isso ocorre por causa da forma pela qual usamos o import.
    - Os próximos slides abordarão as diferentes maneiras de utilizar o import.

```
import mod1
print(mod1.s)
print(mod1.t)
mod1.f_teste(1000)
a = mod1.MinhaClasse()
```

```
>>> %Run testa_mod1.py
Total eclipse of the heart
(5, 10, 15, 20)
x =  1000
objeto instanciado!
```

Modularização no Python (7/15)

- **<u>Forma 1</u>: import <módulo>**
  - Esta é a forma mais simples e segura.
  - Objetos do módulo **não** ficam diretamente acessíveis ao programa que o chamou.
    - O módulo importado cria um namespace separado.
    - **VANTAGEM**: minimiza possibilidade de conflito
    - **DESVANTAGEM**: objetos do módulo só podem ser acessados utilizando a notação *nome_do_modulo.nome_do_objeto*.

```
>>> import math
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'math']
>>> math.sqrt(81)
9.0
```

Modularização no Python (8/15)

- **<u>Forma 2</u>: import <módulo> as <apelido>**
  - Igual à anterior, mas renomeando o módulo para um apelido mais simples.
  - A vantagem é que podemos utilizar um nome menor na hora de prefixar.
    - Muito utilizada com as bibliotecas numpy (np), pandas (pd) e matplotlib (plt).

```
>>> import math as m
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'm']
>>> m.sqrt(81)
9.0
```

Modularização no Python (9/15)

- **<u>Forma 3</u>: from <módulo> import ***
  - Importa os nomes de **todos** os objetos do módulo para o namespace local (com exceção dos que comecem por underscore "_").
  - **VANTAGEM**: não preciso mais prefixar o nome do módulo para acessar o objeto.
  - **DESVANTAGEM**: requer cuidado! Como essa forma de importação coloca os nomes diretamente no namespace, qualquer objeto com o mesmo nome será sobrescrito.

```
>>> from math import *
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2',
'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf',
'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum',
'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt',
'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'perm', 'pi',
'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau',
'trunc']
```

Modularização no Python (10/15)

- **Forma 4: from <módulo> import** nome1, nome2, ...
  - Similar à anterior, mas permite com que você importe nomes do módulo de forma individual.
  - Útil se você sabe que vai precisar apenas de alguns poucos nomes.

```
>>> from math import sqrt, pi, log
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'log', 'pi', 'sqrt']
>>>
```

Modularização no Python (11/15)

- **A variável de sistema __name__**
  - Conforme visto, um módulo Python é simplesmente um programa ".py".
    - Sendo assim, qualquer módulo pode ser executado diretamente:

    > **python** mod1.py

  - O código do módulo "mod1.py" **vai ser executado da mesma forma** se este módulo for importado por outro módulo ou se ele for executado diretamente...
  - ... Porém, com **uma** única **exceção**.
  - Esta é a exceção:
    - No segundo caso (módulo executado diretamente), a variável de sistema **__name__** receberá o valor "__main__".
    - Veja um exemplo no próximo slide… depois explicaremos a utilidade desse recurso.

Modularização no Python (12/15)

- **<u>Caso 1</u>: valor de __name__ dentro do “mod1” quando “mod1.py” é executado <u>diretamente</u>**
  - Nesse caso, o valor é “__main__”

Thonny - C:\ENCE\ProgAvancada\ProgAvan_2024.1\progs\mod1.py @ 12 : 26

File Edit View Run Tools Help

mod1.py

```python
s = "Total eclipse of the heart"
t = (5, 10, 15, 20)

def f_teste(x):
    print('x = ', x)


class MinhaClasse:
    def __init__(self):
        print("objeto instanciado!")

print('__name__ do "mod1":', __name__)
```

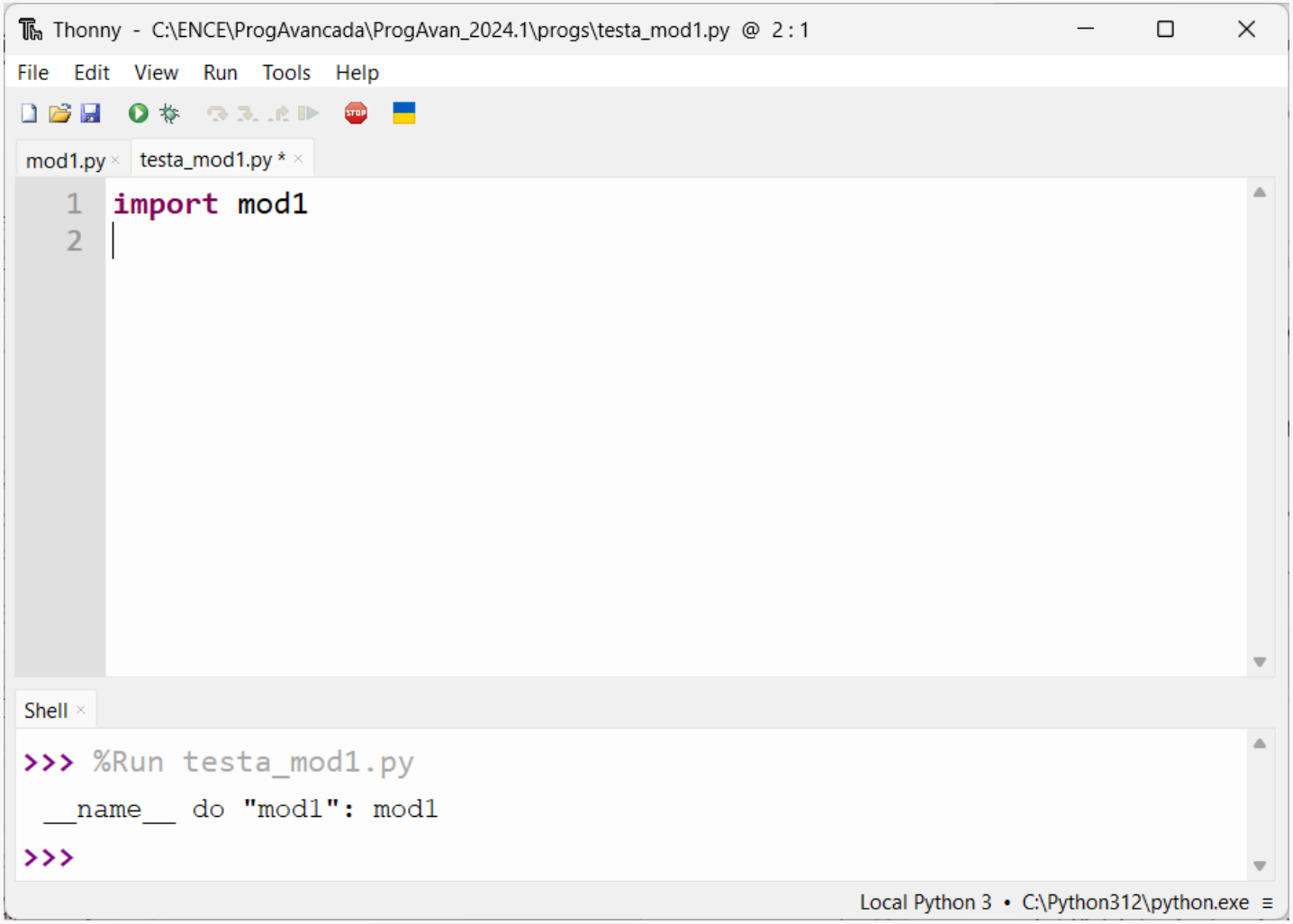
Shell

```
>>> %Run mod1.py
 __name__ do "mod1": __main__
>>>
```

Local Python 3 • C:\Python312\python.exe

Modularização no Python (13/15)

- **<u>Caso 2</u>: valor de __name__ dentro do “mod1” quando “mod1.py” é executado <u>via importação</u>**
  - Nesse caso, o valor é “mod1”



Modularização no Python (14/15)

- **A variável de sistema __name__**
  - Mas para que serve isso????
    - Basicamente, podemos usar __name__ para programar **comportamentos diferentes** em um módulo **para os dois casos** !!!
      - Quando o módulo é executado diretamente.
      - Quando o módulo é executado porque foi importado.
  - Na POO, é muito comum usar esse recurso para colocar o código de teste ou exemplo de uso de uma classe junto ao módulo da classe.
    - Daí colocamos um if __name__ == "__main__" para garantir que o código de teste rode apenas quando o módulo da classe é executado diretamente.
  - Veja um exemplo no slide a seguir

Modularização no Python (15/15)

- **A variável de sistema __name__**
  - Exemplo de código para testar ou demonstrar como usar uma classe

Thonny - C:\ENCE\ProgAvancada\ProgAvan_2024.1\progs\robot.py @ 13 : 28

File Edit View Run Tools Help

robot.py

```python
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2024
        else:
            self.ano_fabricacao = ano_fabr

# código de teste
if __name__ == "__main__":
    x = Robot("Zed")
    y = Robot("Gort", 2008)
    print(x.__dict__)
    print(y.__dict__)
```

Shell

```
>>> %Run robot.py
 {'nome': 'Zed', 'ano_fabricacao': 2024}
 {'nome': 'Gort', 'ano_fabricacao': 2008}
>>> |
```

Local Python 3 • C:\Python312\python.exe

Referências

- Sturtz, J. (2020). Python Modules and Packages – An Introduction. https://realpython.com/python-modules-packages/

- Python 3 Tutorial – Modular Programming and Modules https://www.python-course.eu/python3_modules_and_modular_programming.php

- Python 3 Tutorial – Namespaces https://www.python-course.eu/python3_modules_and_modular_programming.php

- Stack Overflow (2011). How do I clear all variables in the middle of a Python script? https://stackoverflow.com/questions/3543833/how-do-i-clear-all-variables-in-the-middle-of-a-python-script